torch.fft.hfftn#

https://pytorch.org/docs/stable/generated/torch.fft.hfftn.html

Description:

Computes the n-dimensional discrete Fourier transform of a Hermitian symmetric input signal. input is interpreted as a one-sided Hermitian signal in the time domain. By the Hermitian property, the Fourier transform will be real-valued. hfftn()/ihfftn() are analogous to rfftn()/irfftn(). The real FFT expects a real signal in the time-domain and gives Hermitian symmetry in the frequency-domain. The Hermitian FFT is the opposite; Hermitian symmetric in the time-domain and real-valued in the frequency-domain. For this reason, special care needs to be taken with the shape argument s, in the same way as with irfftn().

Function Signature

```
torch.fft.hfftn(input, s=None, dim=None, norm=None, *,
out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> torch.fft.hfftn(t).size()
torch.Size([10, 10])
```

```
>>> roundtrip = torch.fft.hfftn(t, T.size())
>>> roundtrip.size()
torch.Size([10, 9])
>>> torch.allclose(roundtrip, T)
True
```

Notes & Warnings

NOTE Note hfftn()/ihfftn() are analogous to rfftn()/irfftn(). The real FFT expects a real signal in the time-domain and gives Hermitian symmetry in the frequency-domain. The Hermitian FFT is the opposite; Hermitian symmetric in the time-domain and real-valued in the frequency-domain. For this reason, special care needs to be taken with the shape argument s, in the same way as with irfftn().

NOTE Note Some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in the zero-frequency term cannot be represented in a real output and so will always be ignored.

NOTE Note The correct interpretation of the Hermitian input depends on the length of the original data, as given by s. This is because each input shape could correspond to either an odd or even length signal. By default, the signal is assumed to be even length and odd signals will not round-trip properly. It is recommended to always pass the signal shape s.

NOTE Note Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. With default arguments, the size of last dimension should be (2^n + 1) as argument s defaults to even output size = 2 * (last_dim_size - 1)

Detailed Documentation

Documentation

Computes the n-dimensional discrete Fourier transform of a Hermitian symmetric input signal.

input is interpreted as a one-sided Hermitian signal in the time domain. By the Hermitian property, the Fourier transform will be real-valued.

hfftn()/ihfftn() are analogous to rfftn()/irfftn(). The real FFT expects a real signal in the time-domain and gives Hermitian symmetry in the frequency-domain. The Hermitian FFT is the opposite; Hermitian symmetric in the time-domain and real-valued in the frequency-domain. For this reason, special care needs to be taken with the shape argument s, in the same way as with irfftn().

Some input frequencies must be real-valued to satisfy the Hermitian property. In these cases the imaginary component will be ignored. For example, any imaginary component in the zero-frequency term cannot be represented in a real output and so will always be ignored.

The correct interpretation of the Hermitian input depends on the length of the original data, as given by s. This is because each input shape could correspond to either an odd or even length signal. By default, the signal is assumed to be even length and odd signals will not round-trip properly. It is recommended to always pass the signal shape s.

Supports torch.half and torch.chalf on CUDA with GPU Architecture SM53 or greater. However it only supports powers of 2 signal length in every transformed dimensions. With default arguments, the size of last dimension should be (2^n + 1) as argument s

defaults to even output size = 2 * (last_dim_size - 1)

input (Tensor) – the input tensor

s (Tuple[int], optional) – Signal size in the transformed dimensions. If given, each dimension dim[i] will either be zero-padded or trimmed to the length s[i] before computing the real FFT. If a length -1 is specified, no padding is done in that dimension. Defaults to even output in the last dimension: s[-1] = 2*(input.size(dim[-1]) - 1).

dim (Tuple[int], optional) – Dimensions to be transformed. The last dimension must be the half-Hermitian compressed dimension. Default: all dimensions, or the last len(s) dimensions if s is given.

norm (str, optional) –

Normalization mode. For the forward transform (hfftn()), these correspond to:

"forward" - normalize by 1/n

"backward" - no normalization

"ortho" - normalize by 1/sqrt(n) (making the Hermitian FFT orthonormal)

Where n = prod(s) is the logical FFT size. Calling the backward transform (ihfftn()) with the same normalization mode will apply an overall normalization of 1/n between the two transforms. This is required to make ihfftn() the exact inverse.

Default is "backward" (no normalization).

out (Tensor, optional) – the output tensor.

Starting from a real frequency-space signal, we can generate a Hermitian-symmetric

time-domain signal: >>> T = torch.rand(10, 9) >>> t = torch.fft.ihfftn(T)

Without specifying the output length to hfftn(), the output will not round-trip properly because the input is odd-length in the last dimension:

So, it is recommended to always pass the signal shape s.
